

DV033G, Software Testing

Assignment 3

Innehållsförteckning

Innehållsförteckning	1
What we did	1
Part A — Testing the AI feature (in the repo)	1
What we found	1
AI use	2
Judgement	2
What we'd test next	2
Part B - Research Trend Review	2

Lab	3
Pair	Simon Kalitta (sika2400) KiryI Yasny (kiya2400)
Commit	aed0a87ef7dee2e2c56d9e26b541246a56 be5d89

What we did

Part A — Testing the AI feature (in the repo)

To test the `buildPrompt()` method, we implemented three new unit tests, each testing one of the structured parameters. These tests were implemented as parameterized tests to ensure that `buildPrompt()` actually uses the data retrieved from the models. At the same time, we used *Mockito* to mock the models, which allowed us to include verification of a model method call as a requirement for the test to pass. This approach ensures that `buildPrompt()` both calls the model methods and uses the data returned by them.

Unit tests for activities 5.2 and 5.3 were implemented according to the instructions.

Activity 5.4 is very similar to

`testBuildPromptEncapsulatesFreeTextInsideDelimiters()`, as it also contains some form of prompt injection.

For activity 5.5, two tests were implemented to stay in line with the single responsibility principle. The first test ensures that the status code is 200, while the other checks that all JSON fields are present.

What we found

There were no special findings about the code during the implementation of the tests. However, it was somewhat noticeable that testing a simple single functionality (e.g. in activity 5.1) may require several unit tests or even running the same test with different samples of data to ensure that the data is retrieved dynamically.

Testing controllers using *mvc* is also something new, as we had never used it before. Previously, we only tested APIs by manually sending requests from `.http` files, Postman, or similar tools.

AI use

In this assignment, we used AI to create sets of data for parameterized tests.

Judgement

Creating samples of data was one of the uses of AI in this assignment, and we found this way of using AI very helpful, as we did not have to spend time coming up with ideas for fake data and could instead delegate this task to AI. As the precision or correctness of the generated samples was not important, the suggested sets were used as they were.

What we'd test next

Some tests could be implemented to make sure that `fallbackExplanation` correctly returns a string containing the structured fields, similarly to how `buildPrompt()` is tested.

Another opportunity for future testing would be more concrete testing of the `/explain` endpoint, e.g. tests that ensure that the seeker's ID, listing's ID, and explanation text match the expected results.

Part B - Research Trend Review

To investigate current testing trends for AI systems, we compared a practitioner source with an academic source on prompt-injection defenses, since this trend connects directly to Activity 5.4 in our test suite.

For the practitioner source, we used Simon Willison's blog post "Delimiters won't save you from prompt injection"

(<https://simonwillison.net/2023/May/11/delimiters-wont-save-you/>, 2023-05-11).

Willison talks about an official prompt-engineering course from

OpenAI/DeepLearning.AI where they recommend wrapping untrusted data input with delimiters. He also shows that wrapping untrusted input in delimiters can be bypassed by including the delimiter characters in the input itself, which closes the data section early and allows a new instruction to follow. This demonstrates that the model

processes all input as a single token stream, with no structural distinction between instructions and data, so delimiters reduce the attack surface without solving the underlying problem.

As the academic source, we used Greshake et al., “Not What You’ve Signed Up For: Compromising Real-World

LLM-Integrated Applications with Indirect Prompt Injection”

(<https://dl.acm.org/doi/epdf/10.1145/3605764.3623985>, 2023-11-30). This paper takes the same underlying problem and sorts the resulting attacks into distinct categories. These categories include stealing data, spreading the injected prompt to other systems, and corrupting the model’s future outputs. They also demonstrate these attacks against real systems such as Bing Chat and GPT-4-based code tools.

The practitioner source demonstrates the failure directly through a single worked example, which makes the mechanism easy to follow. The academic source generalizes the same failure into distinct categories and verifies these attacks against real production systems, which gives the same conclusion more evidential support.

We used this literature to justify Activity 5.4. Our test,

`testBuildPromptKeepsPromptInjectionInsideDataBlock`, verifies that `buildPrompt()` places untrusted listing-description text inside the delimiter block even when the description contains an injected instruction. The test does not call the LLM; it only checks the structure of the generated prompt string. Based on both sources, this is the correct scope for a deterministic unit test: the structure of the prompt can be verified exactly. Whether a live model actually respects that structure depends on the specific model and remains unresolved, a question a unit test can’t settle.